

Bài Tập Thực Hành AI Tuần 5

Võ Minh Thịnh
MSSV: 22280087

1 Bài toán

Sử dụng thuật toán A^* để giải bài toán TSP và heuristic được sử dụng là cây khung nhỏ nhất.

Traveling Salesperson Problem - TSP: Cho trước n thành phố và các khoảng cách di chuyển giữa mỗi cặp thành phố, tìm tour ngắn nhất sao cho mỗi thành phố được viếng thăm chỉ một lần.

Cây khung nhỏ nhất (Minimum Spanning Tree – MST): Cho đồ thị $G = (V, E)$ là một đồ thị vô hướng với các cạnh có trọng số d_{ij} .

Một cây khung T là một đồ thị con của G thỏa mãn:

- T là một cây (tức là một đồ thị liên thông và không có chu trình).
- T mở rộng tất cả các đỉnh của đồ thị G (bao gồm toàn bộ các đỉnh V của G).

Mỗi cây khung có đúng $(n - 1)$ cạnh, trong đó n là số đỉnh của đồ thị G .

Chiều dài của cây khung: Chiều dài của một cây khung T được định nghĩa là:

$$\text{Chiều dài của } T = \sum_{(i,j) \in T} d_{ij}$$

trong đó d_{ij} là trọng số của cạnh (i, j) thuộc cây khung T .

Bài toán Cây khung nhỏ nhất: Tìm một cây khung T của đồ thị G sao cho tổng trọng số của các cạnh trong T là nhỏ nhất, tức là:

$$\min_T \sum_{(i,j) \in T} d_{ij}$$

2 Cài đặt chương trình

2.1 Class TreeNode

```
1. class TreeNode(object):
2.     def __init__(self, c_no, c_id, f_value, h_value, parent_id):
3.         self.c_no = c_no
4.         self.c_id = c_id
5.         self.f_value = f_value
6.         self.h_value = h_value
7.         self.parent_id = parent_id
```

Định nghĩa một lớp Python có tên là `TreeNode`, đại diện cho một nút (node) trong cây (tree)

- `c_no`: Số thứ tự của nút trong cây.
- `c_id`: ID duy nhất của nút.
- `f_value`: Giá trị của hàm đánh giá $f(n)$.
- `h_value`: Giá trị heuristic của nút.
- `parent_id`: ID của nút cha (*parent*) trong cây.

2.2 Class FringeNode

:

```
1. class FringeNode(object):
2.     def __init__(self, c_no, f_value):
3.         self.c_no = c_no
4.         self.f_value = f_value
```

Lớp này được sử dụng để biểu diễn một nút trong danh sách biên (fringe) của thuật toán tìm kiếm. Mỗi nút bao gồm các thành phần sau:

- `c_no`: Số thứ tự hoặc chỉ số duy nhất của nút trong cây tìm kiếm.
- `f_value`: Giá trị của hàm đánh giá $f(n)$ tại nút này.

2.3 Class Graph

Lớp Graph được thiết kế để biểu diễn đồ thị bằng ma trận kề và thực hiện các thao tác liên quan đến Cây khung nhỏ nhất (MST) bằng thuật toán Prim

```
1. class Graph():
2.     def __init__(self, vertices):
3.         self.V = vertices
4.         self.graph = [[0 for column in range(vertices)]
5.                        for row in range(vertices)]
```

- `self.V`: Số lượng đỉnh trong đồ thị.
- `self.graph`: Ma trận kề (adjacency matrix) để lưu trọng số giữa các cặp đỉnh.
 - Ma trận này có kích thước $V \times V$, với mỗi phần tử `graph[i][j]` biểu thị trọng số cạnh giữa đỉnh i và j .
 - Nếu không có cạnh, giá trị sẽ là 0.

Hàm printMST

```
1. def printMST(self, parent, d_temp, t):
2.     sum_weight = 0
3.     min1 = 10000
4.     min2 = 10000
5.     r_temp = {} # Reverse dictionary
6.     for k in d_temp:
7.         r_temp[d_temp[k]] = k
8.     for i in range(1, self.V):
9.         sum_weight = sum_weight + self.graph[i][parent[i]]
10.        if (graph[0][r_temp[i]] < min1):
11.            min1 = graph[0][r_temp[i]]
12.        if (graph[0][r_temp[parent[i]]] < min1):
13.            min1 = graph[0][r_temp[parent[i]]]
14.        if (graph[t][r_temp[i]] < min2):
15.            min2 = graph[t][r_temp[i]]
16.        if (graph[t][r_temp[parent[i]]] < min2):
17.            min2 = graph[t][r_temp[parent[i]]]
18.    return (sum_weight + min1 + min2) % 10000
19.
```

Hàm `printMST` tính tổng trọng số của cây khung nhỏ nhất (MST) từ ma trận kề và thêm vào hai trọng số nhỏ nhất được kết nối với hai đỉnh cụ thể là đỉnh gốc 0 và đỉnh t . Đầu tiên, hàm khởi tạo các biến: `sum_weight` để lưu tổng trọng số MST, `min1` và `min2` để lưu trọng số nhỏ nhất từ đỉnh 0 và t tới các đỉnh khác, và `r_temp` để tạo ánh xạ đảo từ `d_temp` giúp truy xuất đỉnh dễ dàng. Sau đó, hàm lặp qua các đỉnh (bỏ qua đỉnh 0), cộng trọng số các cạnh giữa đỉnh cha `parent[i]` và đỉnh con i vào tổng trọng số `sum_weight`,

đồng thời cập nhật trọng số nhỏ nhất `min1` và `min2` bằng cách kiểm tra trọng số các cạnh liên quan đến đỉnh `r_temp[i]` và đỉnh cha `r_temp[parent[i]]`. Cuối cùng, hàm trả về tổng trọng số bằng công thức:

$$\text{result} = (\text{sum_weight} + \text{min1} + \text{min2}) \mod 10000.$$

Hàm minKey

```

1. def minKey(self, key, mstSet):
2.     # Initilaize min value
3.     min = sys.maxsize
4.     for v in range(self.V):
5.         if key[v] < min and mstSet[v] == False:
6.             min = key[v]
7.             min_index = v
8.     return min_index

```

Hàm `minKey` tìm đỉnh có giá trị trọng số nhỏ nhất trong mảng `key` và chưa được thêm vào cây khung nhỏ nhất (MST).

Đầu tiên, hàm khởi tạo biến `min` với giá trị lớn nhất có thể (`sys.maxsize`) và `min_index` để lưu chỉ số của đỉnh có trọng số nhỏ nhất. Sau đó, hàm duyệt qua tất cả các đỉnh `v`; với mỗi đỉnh `v`, nếu giá trị trọng số `key[v]` nhỏ hơn giá trị hiện tại của `min` và đỉnh `v` chưa thuộc MST (`mstSet[v] == False`), thì cập nhật `min` thành `key[v]` và `min_index` thành `v`.

Cuối cùng, hàm trả về chỉ số của đỉnh có trọng số nhỏ nhất.

Hàm primMST

```

1. def primMST(self, g, d_temp, t):
2.     key = [sys.maxsize] * self.V
3.     parent = [None] * self.V
4.     key[0] = 0
5.     mstSet = [False] * self.V
6.     sum_weight = 10000
7.     parent[0] = -1
8.
9.     for c in range(self.V):
10.        u = self.minKey(key, mstSet)
11.        mstSet[u] = True
12.        for v in range(self.V):
13.            if self.graph[u][v] > 0 and mstSet[v] == False and key[v] > self.graph[u][v]:
14.                key[v] = self.graph[u][v]
15.                parent[v] = u
16.
17.     return self.printMST(parent, d_temp, t)

```

Hàm `primMST` sử dụng thuật toán Prim để xây dựng cây khung nhỏ nhất (MST) từ đồ thị đã cho và trả về tổng trọng số của MST, bao gồm cả hai trọng số nhỏ nhất liên quan đến các đỉnh cụ thể, được tính bởi hàm `printMST`. Đầu tiên, hàm khởi tạo các cấu trúc dữ liệu:

- **key**: Mảng lưu trọng số nhỏ nhất để kết nối mỗi đỉnh với MST, ban đầu gán giá trị `sys.maxsize`.
- **parent**: Mảng lưu đỉnh cha của mỗi đỉnh trong MST.
- **mstSet**: Danh sách Boolean để đánh dấu các đỉnh đã được thêm vào MST.

Đỉnh gốc 0 được chọn làm điểm bắt đầu với trọng số ban đầu `key[0] = 0` và không có đỉnh cha (`parent[0] = -1`).

Trong vòng lặp chính, hàm chọn đỉnh u có trọng số nhỏ nhất từ mảng `key`, được thực hiện bởi hàm `minKey`. Sau khi chọn được đỉnh u , hàm đánh dấu rằng u đã được thêm vào MST (`mstSet[u] = True`). Tiếp theo, hàm cập nhật mảng `key` và `parent` cho các đỉnh lân cận v của u , nếu các điều kiện sau thỏa mãn:

- `graph[u][v] > 0`: Có cạnh nối giữa u và v .
- `mstSet[v] = False`: Đỉnh v chưa thuộc MST.
- `key[v] > graph[u][v]`: Trọng số cạnh $u-v$ nhỏ hơn giá trị hiện tại của `key[v]`.

Khi tất cả các đỉnh đã được xử lý, hàm gọi `printMST` để tính tổng trọng số của MST và thêm vào hai trọng số nhỏ nhất liên quan đến đỉnh 0 và t . Kết quả trả về được tính bằng công thức:

$$\text{Tổng trọng số} = \text{printMST}(\text{parent}, \text{d_temp}, t).$$

Hàm heuristic

```

1. def heuristic(tree, p_id, t, V, graph):
2.     visited = set()
3.     visited.add(0)
4.     visited.add(t)
5.     if (p_id != -1):
6.         tnode = tree.get_node(str(p_id))
7.         while(tnode.data.c_id != 1):
8.             visited.add(tnode.data.c_no)
9.             tnode = tree.get_node(str(tnode.data.parent_id))
10.    l = len(visited)
11.    num = V - l
12.    if (num != 0):
13.        g = Graph(num)
14.        d_temp = {}
15.        key = 0
16.        for i in range(V):
17.            if (i not in visited):
18.                d_temp[i] = key
19.                key = key + 1
20.        i = 0
21.        for i in range(V):
22.            for j in range(V):
23.                if ((i not in visited) and (j not in visited)):
24.                    g.graph[d_temp[i]][d_temp[j]] = graph[i][j]
25.
26.        mst_weight = g.primMST(g.graph, d_temp, t)
27.        return mst_weight
28.    else:
29.        return graph[t][0]
30.

```

Hàm `heuristic(tree, p_id, t, V, graph)` tính toán một giá trị heuristic dựa trên cây hiện tại và đồ thị đầu vào.

Đầu tiên, nó khởi tạo một tập hợp `visited` để lưu trữ các đỉnh đã thăm, bao gồm đỉnh gốc (0) và đỉnh đích (`t`). Nếu `p_id` (ID của đỉnh cha) khác -1, hàm tiếp tục duyệt từ `p_id` về đỉnh gốc, thêm các đỉnh đã thăm vào tập hợp `visited`.

Sau đó, hàm tính toán số lượng đỉnh chưa thăm (`num`), và nếu không còn đỉnh chưa thăm, trả về trọng số của cạnh nối giữa đỉnh `t` và đỉnh gốc. Nếu vẫn còn đỉnh chưa thăm, hàm xây dựng một đồ thị con chứa các đỉnh chưa thăm và áp dụng thuật toán Prim để tính toán trọng số của cây khung nhỏ nhất (MST) cho đồ thị con này.

Cuối cùng, hàm trả về trọng số của MST hoặc trọng số cạnh từ đỉnh `t` tới đỉnh 0 nếu không còn đỉnh nào chưa thăm.

2.4 Hàm checkPath(tree, toExpand, V):

```
1. def checkPath(tree, toExpand, V):
2.     tnode = tree.get_node(str(toExpand.c_id))
3.     list1 = list()
4.     if (tnode.data.c_id == 1):
5.         return 0
6.     else:
7.         depth = tree.depth(tnode)
8.         s = set()
9.         while (tnode.data.c_id != 1):
10.            s.add(tnode.data.c_no)
11.            list1.append(tnode.data.c_no)
12.            tnode = tree.get_node(str(tnode.data.parent_id))
13.        list1.append(0)
14.        if (depth == V and len(s) == V and list1[0] == 0):
15.            print("Path complete")
16.            list1.reverse()
17.            print(list1)
18.            return 1
19.        else:
20.            return 0
```

Hàm `checkPath` có chức năng kiểm tra xem một cây có chứa một đường đi từ một nút con đến nút gốc hay không.

Nếu đường đi này thỏa mãn các điều kiện nhất định, hàm sẽ in ra đường đi và trả về 1, ngược lại trả về 0. Cụ thể, hàm lấy nút cần kiểm tra từ cây và nếu nút đó là nút gốc, hàm trả về 0 ngay lập tức.

Sau đó, hàm sử dụng một vòng lặp để đi ngược từ nút hiện tại đến nút gốc, lưu các nút trong một danh sách và tập hợp. Độ sâu của cây được kiểm tra để đảm bảo rằng tất cả các nút đều được đi qua.

Nếu các điều kiện độ sâu, số lượng nút trong đường đi và việc bắt đầu từ nút gốc được thỏa mãn, hàm in ra thông báo "Path complete" và đường đi trước khi trả về 1. Nếu không thỏa mãn các điều kiện trên, hàm trả về 0.

2.5 Hàm startTSP(graph, tree, V):

```
1. def startTSP(graph, tree, V):
2.     goalState = 0
3.     times = 0
4.     toExpand = TreeNode(0, 0, 0, 0, 0) # Node to expand
5.     key = 1 # Unique Identifier for a node in the tree
6.     heu = heuristic(tree, -1, 0, V, graph) # Heuristic for node 0 in the tree
7.     tree.create_node("1", "1", data=TreeNode(0,1,heu,heu,-1)) # Create 1st node in the tree :
8.     fringe_list = {} # Fringe list (Dictionary) (FL)
9.     fringe_list[key] = FringeNode(0, heu)
10.    key = key + 1
11.    while (goalState == 0):
12.        minf = sys.maxsize
13.        # Pick node having min f_value from the fringe list
14.        for i in fringe_list.keys():
15.            if (fringe_list[i].f_value < minf):
16.                toExpand.f_value = fringe_list[i].f_value
17.                toExpand.c_no = fringe_list[i].c_no
18.                toExpand.c_id = i
19.                minf = fringe_list[i].f_value
20.
21.        h = tree.get_node(str(toExpand.c_id)).data.h_value # Heuristic value of the selected
22.        val = toExpand.f_value - h # g value of the selected node
23.        path = checkPath(tree, toExpand, V) # Check path of selected node if it is complete
24.        # If node to expand is 0 and path is complete, we are done
25.        # We check node at the time of expansion and not at the time of generation
26.        if (toExpand.c_no == 0 and path == 1):
27.            goalState = 1
28.            cost = toExpand.f_value # Total actual cost incurred
29.        else:
30.            del fringe_list[toExpand.c_id] # Remove node from FL
31.            j = 0
32.            # Evaluate f_values and h_values of adjacent nodes of the node to expand
33.            while (j < V):
34.                if (j != toExpand.c_no):
35.                    h = heuristic(tree, toExpand.c_id, j, V, graph) # Heuristic calc
36.                    f_val = val + graph[j][toExpand.c_no] + h # g(parent) + g(parent->child)
37.                    fringe_list[key] = FringeNode(j, f_val)
38.                    tree.create_node(str(toExpand.c_no), str(key), parent=str(toExpand.c_id))
39.                    key = key + 1
40.                j = j + 1
41.    return cost
```

Hàm `startTSP` là một thuật toán giải bài toán TSP (Traveling Salesman Problem) sử dụng tìm kiếm dựa trên cây, cụ thể là thuật toán A*. Hàm bắt đầu bằng cách khởi tạo một số biến như `goalState` để kiểm tra trạng thái hoàn thành, `times` để đếm số lần lặp (không sử dụng trong đoạn code này), và `toExpand` là nút cần mở rộng, được khởi tạo với thông tin ban đầu. Hàm tính giá trị heuristic (`heu`) cho nút gốc và tạo nút gốc trong cây.

Danh sách `fringe_list`, được sử dụng để lưu các nút chưa được kiểm tra, được khởi tạo dưới dạng từ điển, với khóa là `key` và giá trị là các đối tượng `FringeNode`. Trong vòng lặp chính, hàm chọn nút có `f_value` nhỏ nhất từ `fringe_list`, tính toán giá trị heuristic `h` và `g` cho nút đó, sau đó gọi hàm `checkPath` để kiểm tra nếu đường đi từ nút hiện tại đến gốc là hoàn chỉnh.

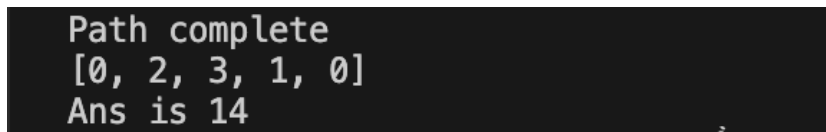
Nếu nút hiện tại là nút gốc (`toExpand.c_no == 0`) và đường đi là hoàn chỉnh (`path == 1`), hàm trả về chi phí của đường đi (`cost`). Nếu chưa hoàn thành, nút hiện tại sẽ được xóa khỏi `fringe_list` và các nút kề sẽ được mở rộng,

với giá trị `f_value` được tính toán và thêm vào `fringe_list`. Vòng lặp tiếp tục cho đến khi đạt được `goalState`, tức là khi tìm được đường đi hoàn chỉnh.

3 Thực thi chương trình

3.1 Kết quả thực thi chương trình

```
1. if __name__ == "__main__":
2.     V = 4
3.     graph = [[0, 5, 2, 3], [5, 0, 6, 3], [2, 6, 0, 4], [3, 3, 4, 0]]
4.
5.     tree = Tree()
6.     ans = startTSP(graph, tree, V)
7.     print("Ans is " + str(ans))
```



```
Path complete
[0, 2, 3, 1, 0]
Ans is 14
```

Path complete: Được in ra khi hàm `checkPath` xác nhận rằng đã tìm thấy một đường đi hoàn chỉnh từ đỉnh gốc đến đỉnh đích.

[0, 2, 3, 1, 0]: Là đường đi từ đỉnh 0 (gốc), qua các đỉnh 2, 3, 1 và quay lại đỉnh 0, tạo thành một chu trình hoàn chỉnh.

Ans is 14: Trọng số của đường đi tối ưu (tổng trọng số của các cạnh trong đường đi).